

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Computer Science and Engineering
ASSESSMENT TOOL 1 – MINI APPLICATION DEVELOPMENT (CHATBOT /
SUMMARIZER / TRANSLATOR)

Course Code:	CSA6501	Course Title:	Generative AI and Large Language Models
CO Assessed:	CO4 – Precision (BL3, BL4,BL5)	Assessment:	Assessment Tool 1-Mini Application Development (Chatbot / Summarizer / Translator)
Weightage:	20% of CIA	Total Marks:	2 * 50= 100
CO4 Statement:	<i>Design and implement multimodal Generative AI applications integrating text, image, and speech models for engineering applications.(BL3, BL4,BL5)</i>		
Student Name:	A.Sai Rohit	Reg. No.:	192472144
Section / Batch:	CSE-AI	Date:	

Instructions to Students

- **Answer any 2 questions. Each question carries 50 marks. Total: 100 marks.**
- Implement the solution using **Python**.
- Select and justify an appropriate AI/LLM model or technique.
- Develop a **working application with a user interface**.
- Test the application using multiple input cases.
- Demonstrate handling of invalid, empty, or unexpected inputs.
- Clearly explain the application architecture and workflow.
- Submit the source code, test results, screenshots/demo, and documentation.
- Explain the limitations of the selected model/technique and suggest possible improvements.

Code of Conduct

I **A.Sai Rohit (192472144)** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: **A.Sai Rohit**

SECTION A – MINI APPLICATION DEVELOPMENT (CHATBOT / SUMMARIZER / TRANSLATOR)

Library Assistant Chatbot – Develop an AI chatbot for a college library that answers questions about book availability, borrowing rules, working hours, membership, and return policies. Include suitable fallback responses for questions outside the available knowledge.

1. AIM

To develop an AI-based chatbot for a college library that answers questions about book availability, borrowing rules, working hours, membership, and return policies. The chatbot also provides suitable fallback responses for questions outside the available knowledge.

2. PROBLEM STATEMENT

Students frequently need information about library services such as working hours, borrowing rules, book availability, and return policies. Getting answers to simple questions may require visiting the library or contacting library staff. The proposed Library Assistant Chatbot provides quick and automatic responses to common library-related questions through a simple web-based interface.

3. OBJECTIVES

- To develop an interactive AI-based library chatbot.
- To answer common library-related questions automatically.
- To use an NLP technique to identify the most relevant question.
- To provide fallback responses for unrelated questions.
- To handle empty, invalid, and unexpected inputs.
- To develop a working web interface that runs on localhost.

4. AI TECHNIQUE USED

The application uses TF-IDF (Term Frequency-Inverse Document Frequency) and Cosine Similarity.

TF-IDF converts the questions stored in the library knowledge base into numerical vectors. The user's question is also converted into a vector. Cosine Similarity is then used to compare the user's question with the stored questions.

The question with the highest similarity score is selected, and its corresponding answer is displayed.

WHY TF-IDF AND COSINE SIMILARITY WERE SELECTED:

- Simple and easy to implement using Python.
- Fast for a small knowledge base.
- Does not require an external API or API key.
- Runs completely on the local computer.
- Easy to understand and explain.
- Suitable for a small domain-specific chatbot.

5. TECHNOLOGIES USED

Programming Language:

Python

Libraries:

- Streamlit
- Scikit-learn

Development Environment:

VS Code

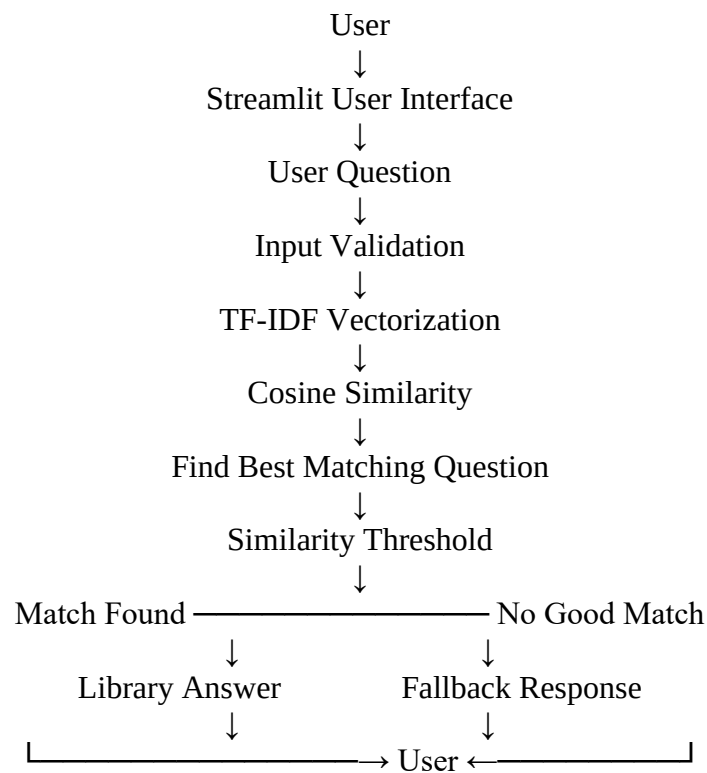
Interface:

Streamlit Web Interface

Deployment:

Localhost

6. APPLICATION ARCHITECTURE



7. WORKFLOW OF THE APPLICATION

1. The user enters a question in the chatbot interface.
2. The application checks whether the input is empty.
3. The user question is converted into a TF-IDF vector.
4. The vector is compared with questions stored in the library knowledge base.

5. Cosine Similarity calculates the similarity between the user's question and stored questions.
6. The question with the highest similarity score is identified.
7. If the similarity score is above the defined threshold, the corresponding library answer is displayed.
8. If the similarity score is below the threshold, the chatbot provides a fallback response.
9. The result is displayed to the user through the Streamlit interface.

8. KNOWLEDGE BASE

The chatbot contains information related to:

- Library working hours
- Library holidays
- Book borrowing rules
- Number of books a student can borrow
- Borrowing period
- Late return policy
- Library membership
- Book availability
- Book renewal
- Return policy

9. IMPLEMENTATION

The application was implemented using Python and Streamlit.

The main libraries used are:

```
import streamlit as st
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity
```

The TF-IDF vectorizer converts the stored questions and user input into numerical representations. Cosine Similarity is used to find the closest matching question.

Code:

```
import streamlit as st
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity

# Library information
knowledge_base = {
    "What are the library working hours?":
        "The library is open from Monday to Saturday, 8:00 AM to 8:00 PM.",

    "Is the library open on Sunday?":
        "No. The library is closed on Sundays and public holidays.",

    "How can I borrow a book?":
```

```

    "You can borrow a book using your valid college library membership card.",

    "How many books can a student borrow?":
        "A student can borrow up to 3 books at a time.",

    "What is the borrowing period?":
        "Books can be borrowed for 14 days.",

    "What happens if I return a book late?":
        "A late fine may be charged according to the library rules.",

    "How can I become a library member?":
        "Students can become members by registering with their student ID at the library.",

    "How can I check book availability?":
        "You can check book availability by entering the book title or asking the librarian.",

    "Can I renew a borrowed book?":
        "Yes, a book can be renewed if another student has not reserved it.",

    "What is the return policy?":
        "Books must be returned on or before the due date to avoid a late fine."
}

```

```

questions = list(knowledge_base.keys())
answers = list(knowledge_base.values())

```

```

# AI model
vectorizer = TfidfVectorizer(stop_words="english")

question_vectors = vectorizer.fit_transform(questions)

```

```

# Function to find answer
def get_answer(user_question):

    # Empty input
    if not user_question.strip():
        return "Please enter a question."

    # Convert question into vector
    user_vector = vectorizer.transform([user_question])

    # Calculate similarity
    similarity = cosine_similarity(
        user_vector,
        question_vectors
    )

```

```

)

# Find best match
best_index = similarity.argmax()
best_score = similarity[0][best_index]

# Fallback for unrelated questions
if best_score < 0.20:
    return (
        "Sorry, I can only answer questions about "
        "book availability, borrowing rules, library hours, "
        "membership, and return policies."
    )

return answers[best_index]

# User Interface
st.set_page_config(
    page_title="Library Assistant",
    page_icon="📖"
)

st.title("📖 College Library Assistant Chatbot")

st.write(
    "Ask questions about library hours, books, "
    "borrowing, membership, and return policies."
)

# Input box
user_question = st.text_input(
    "Enter your question:"
)

# Ask button
if st.button("Ask"):

    answer = get_answer(user_question)

    st.subheader("🤖 Answer")

    st.write(answer)

```

10. Implementation

College Library Assistant Chatbot

Ask questions about library hours, books, borrowing, membership, and return policies.

Enter your question:

What are the library working hours?

Ask

Answer

The library is open from Monday to Saturday, 8:00 AM to 8:00 PM.

College Library Assistant Chatbot

Ask questions about library hours, books, borrowing, membership, and return policies.

Enter your question:

How many books can I borrow?

Ask

Answer

A student can borrow up to 3 books at a time.



College Library Assistant Chatbot

Ask questions about library hours, books, borrowing, membership, and return policies.

Enter your question:

What is the weather today?

Ask

Answer

Sorry, I can only answer questions about book availability, borrowing rules, library hours, membership, and return policies.

11. RESULTS AND DISCUSSION

The developed Library Assistant Chatbot successfully answers common college library questions using TF-IDF and Cosine Similarity. The system identifies the most relevant question from the predefined knowledge base and displays the corresponding answer.

The chatbot successfully handles questions related to library working hours, book borrowing, book renewal, membership, book availability, and return policies.

The fallback mechanism prevents the chatbot from providing unrelated answers when the user asks questions outside the available knowledge. Empty input is also handled properly.

The Streamlit interface provides a simple and interactive user experience and allows the application to run locally through a web browser.

12. LIMITATIONS

- The chatbot works only with information available in its predefined knowledge base.
- TF-IDF does not understand the deeper meaning or context of sentences.
- Questions written using very different wording may sometimes produce an incorrect match.
- The application does not connect to a real-time library database.
- It cannot automatically check the actual availability of books.
- It cannot answer questions outside its predefined domain.
- The system does not maintain real-time student borrowing records.

13. FUTURE IMPROVEMENTS

- Connect the chatbot to a real-time library database.
- Add real-time book availability checking.
- Add student login and personalized borrowing information.

- Use advanced NLP models such as BERT or Sentence Transformers.
- Implement Retrieval-Augmented Generation (RAG).
- Add voice-based question answering.
- Add multilingual support.
- Add automatic due-date and fine calculation.
- Integrate the chatbot with the college library management system.

14. CONCLUSION

The Library Assistant Chatbot provides a simple and effective solution for answering common college library queries. The system uses TF-IDF and Cosine Similarity to identify the most relevant user question and provide an appropriate response.

The application successfully handles valid questions as well as empty, invalid, and unrelated inputs through a fallback mechanism. The Streamlit interface makes the system interactive and easy to use, while localhost deployment allows the application to run without requiring an external server.

Overall, the project demonstrates how basic NLP and AI techniques can be used to develop a practical domain-specific chatbot for college library assistance.

Meeting Minutes Summarizer – Develop an AI application that summarizes meeting transcripts and identifies important decisions, discussions, and action items.

1. AIM

To develop an AI-based application that summarizes meeting transcripts and identifies important decisions, discussions, and action items. The application provides the results through a simple web-based user interface.

2. PROBLEM STATEMENT

Meeting discussions can be lengthy and contain a large amount of information. Important decisions and action items may be difficult to identify from a complete transcript. The proposed Meeting Minutes Summarizer automatically extracts the most important information from a meeting transcript and presents it in a clear and concise format.

3. OBJECTIVES

- To develop an AI-based meeting transcript summarization application.
- To generate a concise summary of a meeting.
- To identify important decisions from the transcript.
- To identify important discussions and action items.
- To handle empty, short, and unexpected inputs.
- To provide a simple and interactive web interface.
- To run the application locally using localhost.

4. AI TECHNIQUE USED

The application uses TF-IDF based Extractive Text Summarization along with rule-based keyword extraction.

TF-IDF is used to identify important sentences in the meeting transcript. Sentences containing important terms receive higher scores and the top-ranked sentences are selected to create the summary.

Rule-based keyword extraction is used to identify important decisions and action items.

Decision-related keywords include:

- decided
- decision
- approved
- agreed
- finalized
- confirmed

Action-related keywords include:

- will
- assigned
- responsible

- complete
- prepare
- submit
- finish
- deadline

WHY THIS TECHNIQUE WAS SELECTED:

- Simple and easy to implement using Python.
- Does not require an external API or API key.
- Runs completely on the local computer.
- Fast for small and medium-sized transcripts.
- Easy to understand and explain.
- Suitable for a mini-project implementation.

5. TECHNOLOGIES USED

Programming Language:
Python

Libraries:

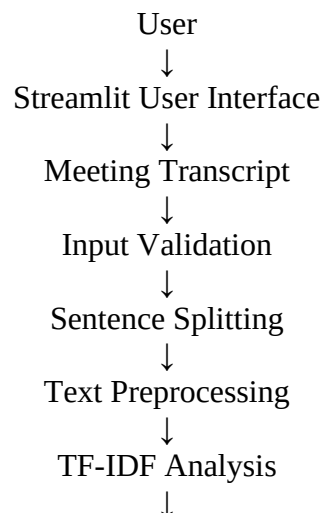
- Streamlit
- Scikit-learn
- Regular Expressions (re)

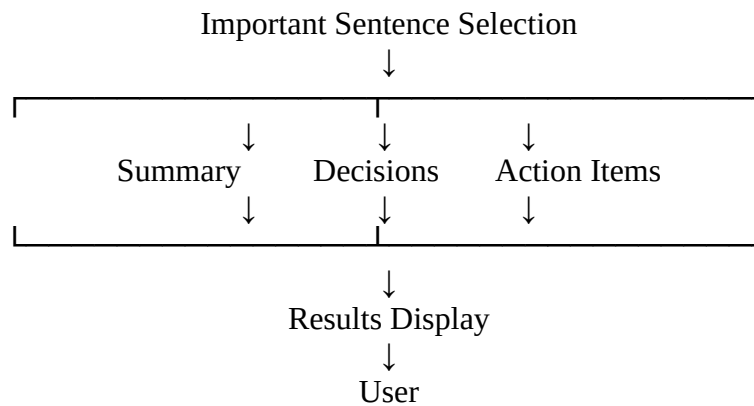
Development Environment:
VS Code

Interface:
Streamlit Web Interface

Deployment:
Localhost

6. APPLICATION ARCHITECTURE





7. WORKFLOW OF THE APPLICATION

1. The user enters or pastes a meeting transcript into the text area.
2. The application checks whether the input is empty.
3. The application checks whether the transcript contains sufficient text.
4. The transcript is divided into individual sentences.
5. TF-IDF is applied to the sentences.
6. Each sentence receives an importance score.
7. The top important sentences are selected to generate the meeting summary.
8. Decision-related keywords are searched to identify important decisions.
9. Action-related keywords are searched to identify action items.
10. The summary, decisions, and action items are displayed through the Streamlit interface.

8. MAIN FEATURES

The application provides three major outputs:

A. MEETING SUMMARY

The system selects the most important sentences from the transcript and presents them as a concise summary.

B. IMPORTANT DECISIONS

The system identifies sentences containing decision-related keywords such as decided, approved, agreed, and confirmed.

C. ACTION ITEMS

The system identifies sentences containing action-related keywords such as will, assigned, responsible, complete, prepare, and submit.

9. IMPLEMENTATION

The application was implemented using Python, Streamlit, and Scikit-learn.

The main libraries used are:

```
import streamlit as st
import re
from sklearn.feature_extraction.text import TfidfVectorizer
```

TF-IDF is used for extractive summarization, while regular expressions are used to divide the transcript into sentences.

Code:

```
import streamlit as st
import re
from sklearn.feature_extraction.text import TfidfVectorizer
```

```
# Split transcript into sentences
def split_sentences(text):
    sentences = re.split(r'(?<=[.!?])\s+', text)

    return [
        sentence.strip()
        for sentence in sentences
        if sentence.strip()
    ]
```

```
# Generate summary
def summarize_text(sentences):

    if len(sentences) <= 2:
        return " ".join(sentences)

    vectorizer = TfidfVectorizer(
        stop_words="english"
    )

    matrix = vectorizer.fit_transform(sentences)

    scores = matrix.sum(axis=1)

    scores = [
        scores[i, 0]
        for i in range(scores.shape[0])
    ]

    # Select top 3 important sentences
    top_indexes = sorted(
        range(len(scores)),
        key=lambda i: scores[i],
        reverse=True
```

```
)[:3]
```

```
# Keep original order  
top_indexes.sort()
```

```
summary = [  
    sentences[i]  
    for i in top_indexes  
]
```

```
return " ".join(summary)
```

```
# Find important decisions  
def extract_decisions(sentences):
```

```
    keywords = [  
        "decided",  
        "decision",  
        "approved",  
        "agreed",  
        "finalized",  
        "confirmed"  
    ]
```

```
    decisions = []
```

```
    for sentence in sentences:
```

```
        if any(  
            word in sentence.lower()  
            for word in keywords  
        ):  
            decisions.append(sentence)
```

```
    return decisions
```

```
# Find action items  
def extract_actions(sentences):
```

```
    keywords = [  
        "will",  
        "assigned",  
        "responsible",  
        "complete",  
        "prepare",  
        "submit",  
        "finish",
```

```

        "deadline"
    ]

    actions = []

    for sentence in sentences:

        if any(
            word in sentence.lower()
            for word in keywords
        ):
            actions.append(sentence)

    return actions


# -----
# User Interface
# -----

st.set_page_config(
    page_title="Meeting Minutes Summarizer",
    page_icon="📝"
)

st.title("📝 AI Meeting Minutes Summarizer")

st.write(
    "Enter a meeting transcript to generate "
    "a summary, decisions and action items."
)


# Text input area
transcript = st.text_area(
    "Enter Meeting Transcript:",
    height=250,
    placeholder="Paste your meeting transcript here..."
)


# Generate button
if st.button("Generate Meeting Minutes"):

    # Empty input
    if not transcript.strip():

        st.warning(
            "Please enter a meeting transcript."

```

)

Very short input

elif len(transcript.strip()) < 20:

```
st.warning(
    "Please enter a longer and meaningful transcript."
)
```

else:

sentences = split_sentences(transcript)

if len(sentences) == 0:

```
st.error(
    "Unable to process the given input."
)
```

else:

summary = summarize_text(sentences)

decisions = extract_decisions(sentences)

actions = extract_actions(sentences)

Summary

st.subheader("💡 Meeting Summary")

st.write(summary)

Decisions

st.subheader("✅ Important Decisions")

if decisions:

```
for decision in decisions:
    st.write("• " + decision)
```

else:

```
st.info(
    "No clear decisions were identified."
)
```

```
# Action items
st.subheader(" 📋 Action Items")
```

```
if actions:
```

```
    for action in actions:
        st.write("• " + action)
```

```
else:
```

```
    st.info(
        "No clear action items were identified."
    )
```

10. IMPLEMENTATION

← → ↺ ⓘ localhost:8501

 **AI Meeting Minutes Summarizer**

Enter a meeting transcript to generate a summary, decisions and action items.

Enter Meeting Transcript:

The team discussed the development of a new student portal.
It was decided that React would be used for the frontend.
Rohit will complete the login module by Friday.
Priya will prepare the database design.
The team agreed to conduct testing next week.
The manager confirmed that the project deadline is next month.

Generate Meeting Minutes

Rohit will complete the login module by Friday.
Priya will prepare the database design.
The team agreed to conduct testing next week.
The manager confirmed that the project deadline is next month.

Generate Meeting Minutes

Meeting Summary

The team discussed the development of a new student portal. Rohit will complete the login module by Friday. The manager confirmed that the project deadline is next month.

Important Decisions

- It was decided that React would be used for the frontend.
- The team agreed to conduct testing next week.
- The manager confirmed that the project deadline is next month.

Action Items

- Rohit will complete the login module by Friday.
- Priya will prepare the database design.
- The manager confirmed that the project deadline is next month.

← → ↺ ⓘ localhost:8501 ☆



AI Meeting Minutes Summarizer

Enter a meeting transcript to generate a summary, decisions and action items.

Enter Meeting Transcript:

Hello everyone.

Generate Meeting Minutes

Please enter a longer and meaningful transcript.

11. RESULTS AND DISCUSSION

The developed Meeting Minutes Summarizer successfully processes meeting transcripts and produces three types of useful information: meeting summary, important decisions, and action items.

The TF-IDF technique successfully identifies important sentences from the transcript and creates an extractive summary. The rule-based keyword extraction identifies sentences related to decisions and action items.

The application also successfully handles empty inputs and very short transcripts by displaying suitable warning messages. When no decisions or action items are detected, the application provides an appropriate message instead of generating incorrect information.

The Streamlit interface provides an easy-to-use environment for entering transcripts and viewing the generated meeting minutes.

12. LIMITATIONS

- The summarizer uses extractive summarization and does not generate completely new sentences.
- TF-IDF does not deeply understand the meaning or context of the meeting.
- Important information may be missed if it does not contain highly relevant terms.
- Decision extraction depends on predefined keywords.
- Action item extraction also depends on predefined keywords.
- The application may incorrectly identify a sentence as an action item if it contains words such as "will" or "complete".
- The application does not identify the speaker automatically.
- It does not automatically extract exact deadlines, names, or responsibilities in a structured format.
- The system may not perform well on very complex or highly conversational transcripts.

13. FUTURE IMPROVEMENTS

- Use advanced transformer models such as BART, T5, or PEGASUS.
- Use large language models for better contextual understanding.
- Add speaker identification.
- Automatically extract task owner, task, and deadline.
- Add structured meeting minutes generation.
- Add support for audio-to-text meeting recordings.
- Add multilingual meeting summarization.
- Add export options such as PDF and Word documents.
- Integrate the system with online meeting platforms.
- Use Retrieval-Augmented Generation or advanced LLM techniques for better contextual results.

14. CONCLUSION

The Meeting Minutes Summarizer successfully provides an automated method for analyzing meeting transcripts. The system uses TF-IDF-based extractive summarization to identify important sentences and rule-based keyword extraction to identify decisions and action items.

The application successfully handles normal meeting transcripts, empty inputs, short inputs, and cases where no clear decisions are available. The Streamlit interface makes the application simple, interactive, and suitable for local deployment.

The project demonstrates how NLP and AI techniques can be applied to reduce the time required to review long meeting transcripts and quickly identify the most important information.

RUBRICS FOR CALCULATION

Criteria	Weight age	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Model/Technique Fit	30%	Correct, appropriate model/technique choice for the chosen application	Mostly appropriate choices made	Some inappropriate technique choices	Technique choice fundamentally flawed
Functional Completeness	25%	Application fully functional; solves a real use-case effectively	Mostly functional application	Partially functional application	Application is non-functional
Robustness & Reliability	20%	Handles edge cases; stable, reliable performance	Handles common cases well	Limited robustness to edge cases	Frequent failures during use
UI / Demo Polish	15%	Polished UI/demo with an engaging walkthrough	Clear demo, minor polish issues	Basic or rough demo	No usable demo presented
Design & Usage Documentation	10%	Comprehensive documentation of design & usage	Adequate documentation	Minimal documentation	No documentation provided

Marks Evaluation:

Question No.	Model/Technique Fit (30%)	Functional Completeness (25%)	Robustness & Reliability (20%)	UI / Demo Polish (15%)	Design & Usage Documentation(10%)	Total Marks (100)
Q1						
Q2						
Overall Total (100)						